

FAQ: Indexes

This document addresses some common questions regarding MongoDB indexes. For more information on indexes, see [indexes](#).

How do I create an index?

To create an index on a collection, use the `db.collection.createIndex()` method. Creating an index is an administrative operation. In general, applications should not call `db.collection.createIndex()` on a regular basis.

NOTE

Index builds can impact performance; see [How does an index build affect database performance?](#). Administrators should consider the performance implications before building indexes.

How does an index build affect database performance?

MongoDB index builds against a populated collection require an exclusive read-write lock against the collection. Operations that require a read or write lock on the collection must wait until the `mongod` releases the lock.

- For feature compatibility version (fcv) "4.2", MongoDB uses an optimized build process that only holds the exclusive lock at the beginning and end of the index build. The rest of the build process yields to interleaving read and write operations.
- For feature compatibility version (fcv) "4.0", the default foreground index build process holds the exclusive lock for the entire index build. `background` index builds do *not* take an exclusive lock during the build process.

For more information on the index build process, see [Index Builds on Populated Collections](#).

Index builds on replica sets have specific performance considerations and risks. See [Index Builds in Replicated Environments](#) for more information. To minimize the impact of building an index on replica sets, including shard replica sets, use a rolling index build procedure as described in [Rolling Index Builds on Replica Sets](#).

How do I monitor index build progress?

To return information on currently running index creation operations, see [Active Indexing Operations](#).

How do I terminate an index build?

To terminate an in-progress index build, use the `db.collection.dropIndex()` or its shell helpers `dropIndex()` or `dropIndexes`. Do not use `db.killOp()` to terminate in-progress index builds in replica sets or sharded clusters.

You cannot terminate a replicated index build on secondary members of a replica set. You must first drop the index on the primary. The primary stops the index build and creates an associated `abortIndexBuild` oplog entry. Secondaries that replicate the `abortIndexBuild` oplog entry stop the in-progress index build and discard the build job.

To learn more, see [Stop In-Progress Index Builds](#).

How do I see what indexes exist on a collection?

To list a collection's indexes, use the `db.collection.getIndexes()` method.

How can I see if a query uses an index?

To inspect how MongoDB processes a query, use the `explain()` method.

How do I determine which fields to index?

A number of factors determine which fields to index, including selectivity, the support for multiple query shapes, and size of the index. For more information, see [Operational Considerations for Indexes and Indexing Strategies](#).

How can I see the size of an index?

The `db.collection.stats()` includes an `indexSizes` document which provides size information for each index on the collection.

Depending on its size, an index may not fit into RAM. An index fits into RAM when your server has enough RAM available for both the index and the rest of the working set. When an index is too large to fit into RAM, MongoDB must read the index from disk, which is a much slower operation than reading from RAM.

In certain cases, an index does not need to fit *entirely* into RAM. For details, see [Indexes that Hold Only Recent Values in RAM](#).

How do write operations affect indexes?

Write operations may require updates to indexes:

- If a write operation modifies an indexed field, MongoDB updates all indexes that have the modified field as a key.

Therefore, if your application is write-heavy, indexes might affect performance.

How does random data impact index performance?

If an operation inserts a large amount of random data (for example, hashed indexes) on an indexed field, insert performance may decrease. Bulk inserts of random data create random index entries, which increase the size of the index. If the index reaches the size that requires each random insert to access a different

index entry, the inserts result in a high rate of WiredTiger cache eviction and replacement. When this happens, the index is no longer fully in cache and is updated on disk, which decreases performance.

To improve the performance of bulk inserts of random data on indexed fields, you can either:

- Drop the index, then recreate it after you insert the random data.
- Insert the data into an empty unindexed collection.

Creating the index after the bulk insert sorts the data in memory and performs an ordered insert on all indexes.